

Adversarially-Robust Inverse Mechanism Design (ARIMD)

A Stackelberg framework for env-design autoresearch

1 Setting

We consider a multi-agent sequential social dilemma (SSD) G with agent set $[N] = \{1, \dots, N\}$, episode horizon T , and per-agent stage rewards $r_t^{(i)} \in \mathbb{R}$. The cumulative return of agent i is $R_i = \sum_{t=1}^T r_t^{(i)}$.

We fix:

- (i) A bounded *edit grammar* $\mathcal{E}_G$ over the environment’s parameters and rule toggles. An element $e \in \mathcal{E}_G$ instantiates a concrete env $G(e)$ that differs from the default G only along the grammar’s listed axes.
- (ii) A frozen *cooperator policy* π_B^{coop} — a hand-crafted reference behaviour, the same across runs.
- (iii) An adversary fraction $\lambda \in [0, 1]$. In every episode the agents are partitioned into a Red subset $\mathcal{R} \subseteq [N]$ with $|\mathcal{R}| = \lfloor \lambda N \rfloor$ (re-sampled per seed) and a Blue complement $\mathcal{B} = [N] \setminus \mathcal{R}$. Red agents play a policy π_R chosen by the adversary; Blue agents play π_B^{coop} .
- (iv) A *welfare functional* Φ that maps the per-agent return vector $\mathbf{R} = (R_1, \dots, R_N)$ to a scalar. Default: Blue-welfare, $\Phi(\mathbf{R}; \mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} R_i$. Population-welfare, $\frac{1}{N} \sum_i R_i$, is reported alongside.

The mixed-population value of an env-policy pair is

$$V_\lambda(e, \pi_R) = \mathbb{E}_{S, \mathcal{R}} [\Phi(\mathbf{R}; \mathcal{B}) \mid G(e), S, \pi_R, \pi_B^{\text{coop}}], \quad (1)$$

where the expectation is over episode seeds S and the random Red-index assignment $\mathcal{R}$. In practice we Monte-Carlo this with a held-out seed grid.

Red is rational and *selfish*: it optimizes its own per-capita return,

$$\Phi_R(\mathbf{R}; \mathcal{R}) = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} R_i. \quad (2)$$

2 The Stackelberg meta-game

Blue commits first to an env design; Red best-responds with the most exploitative homogeneous-Red policy it can find. The *adversarially-robust mechanism* is the solution of

$$e^* = \arg \max_{e \in \mathcal{E}_G} \min_{\pi_R \in \Pi_R} V_\lambda(e, \pi_R), \quad (3)$$

where Π_R is the space of well-formed exploit policies (deterministic functions of the env state returning legal actions; in our implementation, Python policies sampled from a language model and validated by an AST-level safety check).

Algorithm 1 ARIMD-Stackelberg outer loop

Require: env grammar $\mathcal{E}_G$, cooperator π_B^{coop} , adversary fraction λ , eval seeds S , rounds T , Red candidates per round M

Require: LLM oracles $\mathcal{D}$ (Blue) and $\mathcal{X}$ (Red)

```
1:  $e \leftarrow e_0$  ▷ identity patch (default env)
2:  $v^* \leftarrow V_\lambda(e, \pi_B^{\text{coop}})$  ▷  $V_0$  self-play baseline
3:  $\mathcal{H} \leftarrow \emptyset$  ▷ round history shown to  $\mathcal{D}, \mathcal{X}$ 
4: for  $t = 1, \dots, T$  do
    // Red search at frozen Blue env  $e$ 
    5: for  $k = 1, \dots, M$  do
        6:  $\pi_R^{(k)} \leftarrow \mathcal{X}(G(e), \pi_B^{\text{coop}}, \mathcal{H})$ 
        7:  $v_R^{(k)}, v_B^{(k)} \leftarrow \text{EVALMIXED}(e, \pi_R^{(k)}, \lambda, S)$ 
        8: end for
        9:  $k^* \leftarrow \arg \max_k v_R^{(k)}$  ▷ selfish-defector best-response, eq. (2)
        10:  $\pi_R^* \leftarrow \pi_R^{(k^*)}$ 
        // Blue patch proposal
        11:  $\delta \leftarrow \mathcal{D}(G(e), \pi_B^{\text{coop}}, \pi_R^*, \mathcal{H}, \mathcal{E}_G)$ 
        12:  $e' \leftarrow \text{APPLYPATCH}(\mathcal{E}_G, e, \delta)$ 
        // Re-evaluate against the same Red set
        13:  $v_B' \leftarrow \min_k V_\lambda(e', \pi_R^{(k)})$ , estimated over  $S$ 
        14: if  $v_B' > v^*$  then
            15:  $e \leftarrow e'; v^* \leftarrow v_B'$  ▷ accept
        16: end if
        17:  $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\delta, \pi_R^*, v_B', v^*)\}$ 
    18: end for
19: return  $e^* \leftarrow e$ , history  $\mathcal{H}$ 
```

Compared to the symmetric pipeline-minimax $\max_{c_B} \min_{c_R} V_\lambda(c_B, c_R)$ in which both sides choose pipeline configurations, formulation (3) gives Blue and Red *different action spaces*: Blue’s lives in a small combinatorial–continuous grammar, Red’s lives in an open program space. This (a) eliminates inner-loop policy synthesis on Blue’s side, (b) reduces LLM-collusion risk between the two proposers, and (c) yields an interpretable headline artifact (the $\mathcal{E}_G$ -diff at e^*).

Diagnostics. For an accepted env e^* and the Red best-response π_R^* we report three quantities:

- **Invasion barrier** $\lambda^* = \sup\{\lambda : V_\lambda(e^*, \pi_R^*) \geq V_0(e^*, \pi_B^{\text{coop}})\}$ — the largest adversary fraction the mechanism tolerates without falling below the no-attack baseline.
- **Exploitability** $\Delta_{\text{ex}}(J') = V_\lambda(e^*, \pi_R^*) - V_\lambda(e^*, \tilde{\pi}_R)$, where $\tilde{\pi}_R$ is obtained from a fresh Red search of J' extra iterations against frozen e^* . $\Delta_{\text{ex}} \approx 0$ certifies near-equilibrium.
- **Stability gap** $|V_\lambda(e_t, \pi_{R,t}^*) - V_\lambda(e_{t-1}, \pi_{R,t-1}^*)|$ across accepted Stackelberg rounds; monotone decrease evidences convergence.

3 Algorithm

ARIMD instantiates (3) as a finite outer loop with two LLM oracles: a *designer* $\mathcal{D}$ (Blue) and an *exploiter* $\mathcal{X}$ (Red). $\mathcal{D}$ emits a JSON patch δ over $\mathcal{E}_G$; $\mathcal{X}$ emits a Python policy. Both are validated and clamped before evaluation. Acceptance is greedy: a Blue proposal is kept only if it improves the worst-case Red value over *all* Red candidates produced in the round.

The mixed-population evaluator. `EVALMIXED`(e, π_R, λ, S) runs $|S|$ episodes on $G(e)$. Each episode draws a fresh Red index set $\mathcal{R}$ with $|\mathcal{R}| = \lfloor \lambda N \rfloor$ uniformly at random; agents in $\mathcal{R}$ act under π_R and the rest under π_B^{coop} . We report $v_R = \frac{1}{|S|} \sum_S \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} R_i$ (Red’s objective) and $v_B = \frac{1}{|S|} \sum_S \frac{1}{|B|} \sum_{i \in B} R_i$ (Blue’s welfare). $\lambda = 0$ is a special case: π_R is ignored and the run measures pure Blue self-play.

Cost model. For T outer rounds, M Red candidates per round, $|S|$ seeds, and one episode per seed, Algorithm 1 runs $T(M + M)|S|$ episodes plus T Blue LLM calls and TM Red LLM calls — $\mathcal{O}(TM|S|)$ in episodes, linear in LLM calls. No inner policy-synthesis loop appears on Blue’s side, which is the principal source of the cost reduction over symmetric pipeline-minimax variants.

Why a bounded grammar. $\mathcal{E}_G$ exposes a small, semantically meaningful action space to $\mathcal{D}$: numeric knobs (with absolute bounds and type-preserving clamping) and structural toggles (booleans / small categoricals). This is what makes the headline artifact interpretable — e^* is a short JSON patch — and what distinguishes ARIMD’s contribution from black-box parameter search. If, in any given run, $\mathcal{D}$ ’s edits cluster on saturated numeric bounds and ignore the structural toggles, that is itself reportable evidence that the grammar (or the mechanism-design problem on this env) does not require qualitatively novel rules to defend against the chosen Π_R .

4 Implementation

We instantiate ARIMD on three sequential social dilemmas: `nested_commons`, a 16-agent compositional dilemma on a 20×20 grid with intra-clan river cleaning, an inter-clan plaza public good, and a held-inventory raiding sub-game; `cleanup`, the standard 10-agent cleaning-vs-harvesting public-goods env; and `production_economy`, an 8-agent multi-step economy used for grammar plumbing tests but not for the trajectories below. Edit grammars are sized for interpretability: 17 numeric knobs and 3 structural toggles for `nested_commons` (`plaza_occupant_only_bonus`, `plaza_local_clean_gate`, `same_clan_retaliation`); 11 numerics and 1 toggle (`beam_enabled`) for `cleanup`. Numeric ranges are absolute clamps centered on each env’s published defaults; toggles are booleans wired into the env’s `step()` dispatcher and gated on the relevant config flag.

Cooperators are hand-crafted, frozen Python policies, the same across all rounds within a run. `nested_commons` ships two: an *efficiency* cooperator with a static role split (cleaners, harvesters, plaza specialists, one tile per clan), and a *maximin* cooperator with the same instantaneous role mix but with every agent rotating through every role on a Latin-square schedule. `cleanup` ships one default cooperator (an adaptive cleaner-vs-harvester role split parameterized by river waste density).

Both LLM oracles use the framework’s existing inference path. $\mathcal{D}$ (Blue) is `claude-opus-4-7`; $\mathcal{X}$ (Red) is `gemini-3.1-pro-preview`. Red proposals pass an AST-level safety check and a single-episode smoke test before evaluation; failed proposals retry up to three times. All trajectories below use $T = 4$ outer rounds, $M = 4$ Red candidates per round, $|S| = 5$ evaluation seeds, $\lambda = 0.25$, and welfare $\Phi = \text{Blue-mean}$.

5 Experiments

We ran three Stackelberg trajectories, varying the (game, cooperator) pair: (`nested_commons`, `maximin`), (`nested_commons`, `efficiency`), and (`cleanup`, `default`). All other hyperparameters are

Table 1: Headline results across three ARIMD trajectories. V_0 : Blue-mean self-play return on the original env. V_λ^* : Blue-mean return under the worst-case Red over the round’s candidate set, on the final accepted env. λ^* : largest invasion fraction at which $V_\lambda \geq V_0$ on the final env. Accepts: rounds (out of $T = 4$) where Blue’s patch improved over the running best. $U_0 \rightarrow U^*$: Perolat efficiency on the original vs final env in self-play.

Run	V_0	V_λ^*	V_λ^*/V_0	λ^*	Accepts	$U_0 \rightarrow U^*$
nc $\times$ maximin	919.17	639.33	0.70	0.00	0 / 4	14.71 $\rightarrow$ 14.71
nc $\times$ efficiency	945.51	5116.65	5.41	0.25	3 / 4	15.13 $\rightarrow$ 85.73
cu $\times$ default	112.90	144.00	1.28	0.40	3 / 4	1.13 $\rightarrow$ 2.08

identical. Table 1 shows the headline panel; each row is a complete ARIMD run. Per-round patches, Red rationales, and full evaluator outputs are stored under `autoresearch/arimd/runs/`.

Trajectory 1 — nc $\times$ maximin (no improvement). The maximin cooperator runs a Latin-square role rotation, so each agent passes through every role each cycle. Blue *tried* structural toggles in every round (each of the three `nested_commons` toggles appeared in at least one proposal across the four rounds), but every patch was rejected. Mechanism: the toggles’ preconditions interact poorly with role rotation. The local-clean gate requires the collector’s home river to be clean at pickup time, but rotation thins the per-clan cleaner population at any given step, so Blue forfeits its *own* plaza picks. The occupant-only bonus requires multiple Blue agents to be co-located in the plaza, which the rotation does not guarantee. The toggles bite Blue at least as hard as they punish Red. $V_\lambda^* = 639.33$ in Table 1 is the round-0 worst-Red baseline against the unmodified env: Blue made no progress beyond seeing the first attack.

Trajectory 2 — nc $\times$ efficiency (multi-round structural search). The efficiency cooperator’s static specialists keep cleaners co-located with their home quadrants and maintain ≥ 3 Blue agents in the plaza. Round 0 (V_λ after patch = 292; rejected) flipped all three toggles aggressively. Round 1 ($\rightarrow 946$; accepted) kept `same_clan_retaliation` and `plaza_occupant_only_bonus` on, with the raid economy locked down (`raid_cost` maxed, `raid_success_prob` minned, held-apple stream zeroed). Round 2 ($\rightarrow 2732$; accepted) is the qualitatively interesting move: Blue *toggled* `plaza_occupant_only_bonus` *back off* after observing Red parking in plaza to harvest the now-occupant-only payouts. Blue’s recorded rationale: “*Disabling occupant-only restores global welfare leakage back to Blue; enabling `plaza_local_clean_gate` ties plaza payouts to the collector’s own river hygiene, which Blue maintains by role and Red ignores.*” Round 3 ($\rightarrow 5117$; accepted) is numeric polish over the round-2 structural choice. Raids per episode collapsed from ~ 15 to 0.0 from round 2 onward. Of the 12 numerics edited in the final patch, 8 are at a grammar bound; the other 4 are within 20% of one — the binding constraint on the numeric component is the grammar, not the LLM.

Trajectory 3 — cu $\times$ default (single-shot structural insight). Cleanup’s only structural toggle is `beam_enabled`; turning it off disables tagging entirely. Round 0 turned it off on the first attempt, with the rationale: “*Blue’s cooperator never fires the beam, so disabling tagging is a free structural win — it removes the asymmetric weapon Red uses to suppress the worst-off Blue harvester.*” Round 3 reaffirms the choice: “*Beam stays off — the frozen cooperator never fires it, so enabling it only arms Red.*” Rounds 2 and 3 are numeric refinement (`clean_cost` slashed, `threshold_restoration` maxed, `apple_respawn_prob` doubled) over the same toggle-off baseline;

2 of the 5 accepted numerics fully saturate. Round 1 was lost to a JSON-parsing failure on Blue’s response and the framework correctly fell back to the identity patch — at $T = 4$ this represents one wasted Blue turn out of four. The $\lambda^* = 0.4$ invasion barrier is the most permissive of the three runs.

Hypothesis status. The plan registered four pre-experiment hypotheses; we summarize their status with the caveat that $N = 1$ trajectory per (game, cooperator):

- **H1 (existence of robust mechanisms):** mixed. Two of three runs lifted V_λ^* above the round-0 baseline; one ran flat.
- **H2 (fairness > welfare for invasion robustness):** *inverted under our grammar.* The efficiency cooperator strictly outperformed the maximin cooperator on `nested_commons`. The new toggles’ role preconditions favor static specialists and break under Latin-square rotation — robustness here is a property of the joint (cooperator, mechanism), not of the cooperator alone.
- **H3 (structural rule edits emerge):** *yes on the two non-trivial runs.* Final accepted patches each carry at least one toggle, and Blue uses the toggle action-space deliberately, including a within-trajectory reversal on `nc` $\times$ efficiency round 2. Caveat: structural toggles *ride on top of* numeric saturation rather than substituting for it; the cleanest H3 falsifier (toggles-off ablation) has not yet been run.
- **H4 (cross-env transfer):** not directly tested — each env’s grammar is bespoke. The same *kind* of toggle move (asymmetric-rule disable) appears across both successful runs.

Efficiency-metric anomaly. The Perolat efficiency $U = \frac{1}{T} \sum_{i,t} r_t^{(i)}$ computed against the *final* accepted env lifts substantially in both successful runs: $15.13 \rightarrow 85.73$ ($5.67\times$) on `nc` $\times$ efficiency, $1.13 \rightarrow 2.08$ ($1.84\times$) on `cu`. The patched env is therefore strictly richer in self-play, not only better at resisting invasion. More striking: on the final `cu` env, U *grows monotonically* with λ ($U \in \{2.08, 2.65, 2.87, 2.87, 3.42\}$ at $\lambda \in \{0, 0.1, 0.2, 0.25, 0.4\}$); on the final `nc` env it is flat across $\lambda \in [0, 0.25]$ and dips slightly only at $\lambda = 0.4$. Mechanism: on `cu`, with `beam_enabled = False`, Red contributes apple harvest with no friendly-fire downside while the cooperator under-harvests on the apple-rich post-patch env. On `nc` the symmetric public-goods plaza channel renders exploitation and participation operationally identical. This is the *saturated public-good* failure mode: once Blue’s edits make the env rich enough, exploit and cooperate become indistinguishable. Cooperator-relative efficiency — the fraction of theoretical per-step ceiling realized by the population — would be a more discriminating headline metric than absolute U .

6 Discussion

What the trajectories show. The framework is mechanically sound and surfaces interpretable artifacts: each successful run’s final $\mathcal{E}_G$ -diff is a short, human-readable patch with at least one structural toggle and a small set of numeric edits. Blue’s recorded rationales correctly identify cooperator–Red asymmetries (the BEAM monopoly on `cleanup`) and adapt non-monotonically when Red exploits a previous Blue choice (the round-2 reversal on `nc` $\times$ efficiency). These behaviors are not guaranteed by the algorithm — they depend on the proposer’s reasoning quality — and serve as positive evidence that the formulation rewards mechanism-design reasoning rather than blind parameter search.

Cooperator–mechanism interaction. The maximin-vs-efficiency divergence on a single grammar is the panel’s most paper-worthy observation. Standard fairness vs. welfare arguments would predict the maximin cooperator to be more invasion-robust under selfish-defector Red; we observe the opposite. The mechanism is structural, not statistical: the toggles’ preconditions (per-clan cleanliness, plaza co-location) are satisfied by static specialists and broken by role rotation. This is precisely the interaction the ARIMD formulation is designed to surface — robustness as a joint property of cooperator and rule-set, not of either in isolation.

What the trajectories do not yet show. With $N = 1$ outer trajectory per cell, every quantitative claim is single-sample. The single critical missing experiment is the *numeric-only ablation*: re-run each successful trajectory with toggles disabled in $\mathcal{E}_G$. If $V_\lambda^\star$ recovers to within statistical noise of the toggles-on result, H3 is falsified — the toggles were decoration on a numeric saturation that did the actual work. The ablation is cheap (a one-line grammar change, identical loop) and is the highest-information next experiment.

7 Limitations and threats to validity

- **Single trajectory per cell.** No outer-loop seed variation. Cleanup’s round-0 structural insight in particular could be lucky — a different rng seed might produce a Red that does not exhibit the BEAM monopoly Blue exploited in round 0.
- **Numeric saturation.** 8/12 accepted numerics on `nc` $\times$ efficiency and 2/5 on `cu` sit at a grammar bound; the bounds, not the LLM, are the binding constraint on the numeric component of the search.
- **Self-play U lift.** The final env’s U_0 exceeds the original env’s U_0 by 1.8–5.7 $\times$ on the successful runs, consistent with Blue *richening* the env rather than only hardening it. Reporting cooperator-relative U , or $V_\lambda - V_0$ at fixed grammar, would distinguish the two regimes.
- **Single LLM oracle pair.** Blue = Opus-4.7, Red = Gemini-3.1-pro. A different oracle assignment could swap which side is more imaginative; we have no ablation here.
- **Parse fragility.** One JSON-parse failure on `cu` round 1 cost 25% of the search budget at $T = 4$. Structured-output mode for $\mathcal{D}$ would close this.
- **Diagnostic.** The stability-gap diagnostic of §2 returns 0.000 on the two successful runs despite per-round V_λ jumps of ~ 1800 reward; the implementation is buggy and the reported values are not load-bearing.

8 Conclusion

ARIMD instantiates a Stackelberg meta-game between an LLM env-designer and an LLM exploit-policy synthesizer over a bounded grammar of mechanism rules. We presented the formalization, the loop algorithm, and three small-scale empirical trajectories. The framework recovers structural-rule edits (`beam_enabled` disabled on `cleanup`; `plaza_local_clean_gate` flipped on `nested_commons`) with interpretable Blue rationales, and surfaces a non-trivial cooperator–mechanism interaction on a single grammar. The critical near-term experiment is the toggles-off ablation; until that is run, the present results are encouraging but not conclusive evidence that the structural action-space dominates parameter saturation.